CODA-MUR: Certified Representation Transport, Composition, and Irreversible Forgetting

Subtitle: A commuting square is not target standing

Author: Jeremy H. Carroll

Date: 2026-09-28

Version: v3.1.1

License: Creative Commons Attribution 4.0 International

CODA Architecture & Layers

CODA stands for the Calculus of Obstruction, Descent, and Authorization. It is an exact, finite mathematical apparatus that enforces strict claim discipline by separating linear repairs, transport certificates, and finite descent from ungrounded authority claims. The system comprises eight operational layers governed by a single overarching authority architecture (GTLA).

Layer	Abbreviation	Layer / Component Name & Scope
1	**FOC**	Finite Obstruction Calculus (exact quotient diagnostics, dual witnesses)
2	**MUR**	Meta & Universal Re-Presentation (what presentations exist and how they transport)
3	**CS**	Continuum Shadow (what continuum presentation may descend to a finite calculus system)
4	**CS_Q**	Continuum Shadow to Continuum Bridge (how continuum presentations descend)
5	**Q**	The Continuum (continuum presentations via completion and enclosure)
6	**A**	Anomalon (global residuals not removed and typed instead)
7	**OR**	Omniversal Re-Presentation (what presentations can transport)
8	**FDA**	Finite Descent Authority (what presentation may descend to evaluation and authorization)
9	**GTLA**	Overarching Layered Authority Architecture for CODA

Abstract

A transport of a picture is a commuting square: move the states, then look, or look, then move the observations. The two routes agree. Two such squares paste: the composite move still commutes. If two states already look the same, they still look the same after the move — collisions travel. A later function of the observation cannot split a collision either. Collapsing two values onto one point cannot

be inverted: no left inverse recovering every source value exists. The square is a certificate that the move is compatible with looking. It is not, by itself, a standing record in the target.

Introduction

It is easy to treat a coarser picture as the same picture. Integers go to parity; a high-resolution grid goes to a low-resolution grid (Cousot and Cousot [5]); a compiled binary is treated as the verified source (Leroy [6]). The move may be honest, and still forget something (Shannon [4]). Two different integers become the same bit. Once they have collided, no later function of that bit can tell them apart.

A picture. Fine states are integers $X = \mathbb{Z}$. Looking at an integer is the identity, or, more usefully, some observation $p : X \to A$. A coarser picture has states $Y = \{0, 1\}$ and a look $q : Y \to B$. The move of states is $F(n) = n \bmod 2$. The move of observations is the matching map on whatever p recorded. Compatibility is the equality

$$q(F(n)) = \bar{F}(p(n))$$

for every n: look after the move, or move after the look. That equality is the transport certificate χ_F (Mac Lane [1]). It is a commuting square, not a decision in the coarser world. In the companion finite obstruction calculus (CODA-FOC [2]) and finite descent authority (CODA-FDA [3]), representation transport connects source and target regimes without laundering unauthorized claims.

The theory is refuted by any of: a pair of commuting squares whose composite fails to commute; a collision $p(x) = p(y)$ that is split by a claimed transport; a post-processing map that separates a collision; a claimed left inverse that recovers both bits from a collapsed point; or a commuting square that is treated, by itself, as target standing.

Notation

Throughout, the types are ordinary (they need not be finite).

- X is a type of states; A is a type of observations of those states.
- $p : X \to A$ is a look: each state is sent to what you record of it.
- Y and B are a coarser type of states and a coarser type of observations.
- $q : Y \to B$ is the coarser look.
- $F : X \to Y$ moves states; $\bar{F} : A \to B$ moves observations.
- A collision is a pair $x, y \in X$ with $p(x) = p(y)$ and, typically, $x \neq y$.
- In the regime language of the companion, $X = \mathsf{Rep}_A$ is a representation, $A = \mathcal{C}_{\mathrm{fin},A}$ is a finite core, $p = D_A$ is descent, F is the representation map, and $\bar{F}$ is the core action. The same square is then written $\chi_F : D_B F \Rightarrow \bar{F} D_A$.
- Target standing, as in the companion finite-descent paper (CODA-FDA [3]), is a pair: a terminal actually produced by the target pipeline, together with the target halt guards. It is not a field of the square.

A **transport** from (X, A, p) to (Y, B, q) is a triple $(F, \bar{F}, \chi)$ whose χ is the commuting equality below.

1. Transports as commuting squares

Definition MUR-D1 — transport

A transport from $p : X \to A$ to $q : Y \to B$ is a record with three fields:

- states : $X \to Y$,

- observations : $A \to B$,
- commutes : $\forall x,\ q(\text{states}(x)) = \text{observations}(p(x))$.

The Lean record is `Coda.MUR.Transport`. In diagrams (Mac Lane [1]):

$$\begin{array}{ccc} X & \xrightarrow{\text{states}} & Y \\ \downarrow p & & \downarrow q \\ A & \xrightarrow{\text{observations}} & B. \end{array}$$

The square commutes if and only if looking after the move equals moving after the look.

Definition MUR-D2 — descent-square certificate χ_F

Let $F : \mathsf{Rep}_A \to \mathsf{Rep}_B$ be a representation map and $\bar{F} : \mathcal{C}_{\text{fin},A} \to \mathcal{C}_{\text{fin},B}$ a map of finite cores, with descent D_A and D_B. A descent-square certificate is the equality

$$\chi_F : \qquad D_B(F(x)) = \bar{F}(D_A(x)) \qquad \text{for every } x.$$

The Lean record is `DescentSquareCertificate`, with field `commutes`. Definition MUR-D1 and Definition MUR-D2 are the same square, written once with states and observations and once with representations and cores.

In the integer-to-parity example, $F(n) = n \bmod 2$. If D_A is "record the integer" and D_B is "record the bit", the matching $\bar{F}$ is $\bar{F}(n) = n \bmod 2$, and χ_F holds. If someone declares a different core action — say $\bar{F}$ constantly 0 — then at every odd n the square fails.

A morphism in the companion is also typed. The recorded kinds are exact, embedding, quotient, approximation, and unmeasured. The last three are lossy: they require a loss ledger (Section 6). The kind is a label on the map. It is not a proof that the square commutes.

2. Composition of two squares

Given transports

$$(X, A, p) \xrightarrow{f} (Y, B, q) \xrightarrow{g} (Z, C, r),$$

the composite has state map $g.\text{states} \circ f.\text{states}$ and observation map $g.\text{observations} \circ f.\text{observations}$.

Theorem MUR-T1 — composite commutes

For every $x \in X$,

$$r(g.\text{states}(f.\text{states}(x))) = g.\text{observations}(f.\text{observations}(p(x))).$$

Proof. Rewrite the left-hand side by g.commutes at f.states(x), then by f.commutes at x. This is Lean `Coda.MUR.composite_commutes`. □

In the regime language the same pasting is

$$D_C(G(F(x))) = \bar{G}(D_B(F(x))) = \bar{G}(\bar{F}(D_A(x))).$$

Theorem MUR-T2 — descent squares paste

If χ_F says $D_B \circ F = \bar{F} \circ D_A$ and χ_G says $D_C \circ G = \bar{G} \circ D_B$, then

$$D_C \circ (G \circ F) = (\bar{G} \circ \bar{F}) \circ D_A.$$

Proof. For each x,

$$D_C(G(F(x))) = \bar{G}(D_B(F(x))) = \bar{G}(\bar{F}(D_A(x))).$$

This is Lean `descentSquareCertificate_paste`. □

Theorem MUR-T1 is the reconstruction form; Theorem MUR-T2 is the same equality on descent squares. Two commuting squares paste. That is not a theorem that every family of regimes is a bicategory: associators, unitors, and interchange are not supplied here, and no such coherence result is claimed.

The companion pastes the Python records in the same direction. Kinds compose by an explicit associative total-order monoid (exact is the two-sided identity, unmeasured absorbs; approximation absorbs quotient; two embeddings remain an embedding; two exact maps remain exact). Loss masses add in declared common units (Section 6). A regime mismatch — the target of F is not the source of G — or an invalid leg is refused before any composite is built.

3. Collisions travel

A collision is two states that already look the same.

Theorem MUR-T3 — collisions transport

Let f be a transport from p to q. If $p(x) = p(y)$, then

$$q(f.\text{states}(x)) = q(f.\text{states}(y)).$$

Proof. The left-hand side is $f.\text{observations}(p(x))$ by commutativity, the right-hand side is $f.\text{observations}(p(y))$, and $p(x) = p(y)$. This is Lean `Coda.MUR.collision_transports`. □

In words: if you could not tell x from y before the move, you cannot tell their images apart after it either. The coarser look q still sees a collision. Parity of 2 equals parity of 4; any commuting transport of that look still reports a match.

Theorem MUR-T4 — post-processing preserves collisions

Let $p : X \to A$ and $f : A \to B$. If $p(x) = p(y)$, then $f(p(x)) = f(p(y))$.

Proof. Apply f to both sides of the assumed equality. This is Lean `Coda.MUR.postprocess_preserves_collision`. □

A later function of the observation cannot split a collision. If two integers have already been recorded as the same bit, hashing that bit, thresholding it, or feeding it to another map still yields one value, not two.

The two theorems are the same idea at two distances from the look: first the transported look $q \circ \text{states}$, then an arbitrary function of p itself. Irreversible forgetting begins here. Distinctions that p has already dropped are not restored by a commuting move, and they are not restored by processing the record.

4. What you cannot get back

Once a fibre has been collapsed to a point, there is no map out of that point that recovers the forgotten bit.

Theorem MUR-T5 — non-recoverability after non-injective collapse

There does not exist a recovery map $\mathrm{recover} : \mathbf{1} \to \mathbf{2}$ such that $\mathrm{recover}(\mathrm{collapse}(b)) = b$ for every bit b.

Proof. Suppose such a `recover` exists. Then $\mathrm{false} = \mathrm{recover}(*) = \mathrm{true}$. This is Lean `Coda.MUR.loss_is_not_recoverable`. □

The statement is deliberately small. The unit type has one point. A function of that point returns one bit. It cannot equal both bits. Any quotient that identifies two states, then discards the quotient class as well, is in this situation: the collapsed fibre is a point, and no left inverse recovering every source value exists.

In the integer-to-parity example the forgotten datum is the magnitude. No function of the bit 0 returns both 2 and 4. In a further collapse that sends even and odd to a single token, even the bit is gone. Theorem MUR-T5 is that last step, written with the smallest types that make the obstruction visible.

A commuting square records that the collapse is compatible with looking. It does not supply an inverse.

5. A commuting square is not target standing

Standing in a target, as in the companion finite-descent paper (CODA-FDA [3]), is a dependent pair: a terminal t that the target pipeline actually produced, together with the target halt guards of t. Write $\mathsf{Standing}_B(y)$ for that pair at a coarser state y. A transport certificate χ_F has no field for t and no field for the guards.

Theorem MUR-T6 — χ alone does not imply target standing

It is not the case that for every target, every coarser state y, and every commuting square χ,

$$\chi \implies \mathsf{Standing}_B(y).$$

Proof. Take a target that never terminalizes: the terminalizer returns nothing at every input, while the guard predicate is constantly true (`noTerminalRegime`). Let χ be the concrete commuting square from $\mathbf{2}$ to $\mathbf{1}$ ($p = \mathrm{id}$, $q = \mathrm{id}$, states $= \mathrm{const}(*)$, observations $= \mathrm{const}(*)$), which commutes definitionally. Target standing asks for some t with $\mathrm{terminalize}(y) = \mathsf{some}(t)$. There is none. This is Lean `CODA_MUR.concrete_commuting_square_uninhabited_standing` and `chi_alone_does_not_imply_target_authority`, with the countermodel `noTerminalRegime`. □

The countermodel is tiny on purpose. A commuting square can hold — χ can be verified — and the target can still refuse to produce a terminal. In the companion, that refusal is `R_noterminal`. If a terminal is produced but fails the target guards, the refusal is `R_target_unauth`. If the square itself does not commute, the refusal is `R_notransport`. Those are three different failures. The square is a compatibility certificate. It is not target standing.

Unfolding the target pair is definitional: target standing implies that some terminal was produced, and that the target guards hold of it. Lean records this as `targetAuthority_unfolds`. The implication is a projection, not a construction of standing from χ_F.

6. Loss ledgers for lossy moves

A lossy kind — quotient, approximation, or unmeasured — may erase, identify, approximate, or leave unresolved some of what the source look recorded. The companion stores that account as a finite ledger.

Definition MUR-D3 — loss ledger

A ledger is a four-bit record (e, i, a, u) together with a non-negative rational mass: erasure, imprecision, approximation, unmeasured, and loss_mass. The empty ledger has all bits false and mass 0. Sum is bitwise or on the bits and addition on the mass. This is the Python and paper record. Lean `CODA_MUR.LossLedger` contains only the four Boolean fields; no rational mass is present in that formal record.

Quotient, approximation, and unmeasured require a non-empty ledger matching the required loss dimension: quotient requires e = true, approximation requires a = true or i = true, and unmeasured requires u = true. Exact and embedding require the ledger to be absent or empty.

Theorem MUR-T7 — loss ledger necessity (policy admissibility)

If a move is of a lossy kind and the ledger is missing or empty, the move is not loss-accounting admissible.

Proof. Loss-accounting admissibility, in the companion kernel, is the policy implication "if loss is required, then `lossDefined` holds". To model a missing or empty runtime ledger, instantiate that proposition as false. The Lean theorem does not itself inspect a ledger or enforce kind-specific bits. The antecedent holds and the consequent fails. This is Lean `loss_accounting_admissible_necessity` (aliased as `loss_ledger_necessity`). □

In the calculator, a quotient certificate with a missing or empty ledger is refused as `R_unpriced_loss` before any target terminal is asked for. The integer-to-parity map is a quotient: it must record that magnitudes were identified.

Theorem MUR-T8 — loss sums associate

Ledger sum is associative, with the empty ledger as a two-sided unit.

Proof. Bitwise or on four bits is associative and unital at `false`; addition of rationals is associative and unital at 0. The Boolean part is Lean `loss_sum_assoc`, `loss_sum_left_unit`, and `loss_sum_right_unit`. The rational extension is the ordinary additive monoid of nonnegative rationals and is tested in Python; that mass component is not mechanized in this package. □

For composable certificates F then G, the companion takes the sum of the two ledgers as the composite ledger while preserving per-leg receipts. Combined with Theorem MUR-T2, two lossy commuting squares paste to one commuting square whose recorded loss is the sum of the recorded losses. Loss does not reset at the join.

The four bits are a finite Boolean algebra of labels, not an inventory of every way a map can forget. Richer identification closures and approximation composition are not proved here.

7. Computational companion

The production module `code/coda_mur_core.py` records descent-square certificates, pastes them, accumulates ledgers, and evaluates a target terminalizer only after the square commutes, target regimes bind, and any required ledger is present. Arithmetic on loss mass is exact rational; floats and booleans in scalar position raise `TypeError`. An optional leftover of a finite linear cochain problem may be attached

as a transport obstruction via the vendored standalone solver `code/cochain_presentation.py`; a positive leftover is `R_obstruction`. That leftover calculator measures leftover. It is not a theorem about the origin of standing.

The verified Python path requires a nonempty finite carrier and all four maps. Boolean flags can veto a record but cannot make a flags-only record verified. The square is checked on the carrier and again on the requested source; the transported artifact must equal the evaluated state map. An optional action must agree with that same value. Thus requested-source checks cover inputs outside the listed carrier without claiming universal correctness of arbitrary callbacks.

An attached quotient certificate must match both the explicitly declared defect cochain and repair matrix. Returned quotient records are frozen, with immutable matrix values. The callbacks and their intended meaning remain part of the declared model; finite equality checks do not establish external semantic truth.

Lean files in `lean4/`:

- `D01D02D03Records.lean` — kinds, ledgers, and the commuting square of Definition MUR-D2;
- `T01CertificatePasting.lean` — Theorem MUR-T2;
- `T03T04T05LossNoCreation.lean` — Theorems MUR-T7 (loss accounting admissibility) and MUR-T8;
- `L02TerminalNonBypass.lean` — Theorem MUR-T6 (concrete countermodel);
- `Algebra.lean` — self-contained reconstruction algebra for Theorems MUR-T1 and MUR-T3–T5. The four symbols are listed separately below;
- `Core.lean` — re-exports the files above.

The reconstruction symbols, all in namespace `Coda.MUR`, are:

- `composite_commutes`;
- `collision_transports`;
- `postprocess_preserves_collision`;
- `loss_is_not_recoverable`.

There are no `sorry` stubs in those files. The reproducibility gate builds the modules with `lake build` using the standard Lean 4 kernel with zero external mathlib or monorepo path escapes. They do not prove a bicategory of all regimes.

Adversarial tests in `code/tests/` include: a missing or non-commuting certificate; target regime mismatch; source regime mismatch; substituted target artifact; pasting invalid legs; quotient with an empty or mass-only ledger; a commuting square whose target never terminalizes; a commuting square whose target terminal fails the target guards; 5×5 kind monoid identity and associativity across all 125 triples; verified quotient distance certificates; float and non-boolean rejection; and static notebook checks. Actual notebook replay runs separately through `verification/replay_notebooks.py` in independent host processes and pinned Python kernels. Counts and outcomes are recorded by the release verification gate.

8. Limitations

The core mathematical theorems are elementary: MUR-T1, T3, and T4 are equality-congruence results; MUR-T5 is the two-point/one-point left-inverse obstruction; T8 is associativity of a declared record operation. The contribution is their packaging into a verified representation transport and target standing firewall, not new category theory or a general theory of information loss.

The reconstruction theorems are equalities and a non-recoverability obstruction after non-injective collapse. They hold for ordinary functions of types. They do not require finiteness.

The companion ledgers and kinds are finite labels. They do not classify every possible information loss.

A bicategory of all representation regimes has been sketched in companion notes. No bicategory axioms, associator, unitor, or interchange theorem for that object is proved here. What is proved is pasting of two commuting squares, transport of collisions, non-recovery from a collapsed point, and the countermodel that a verified χ need not inhabit target standing.

References

1. S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Graduate Texts in Mathematics, vol. 5, Springer, 1998. https://doi.org/10.1007/978-1-4757-4721-8
2. J. H. Carroll, *The Finite Obstruction Calculus: Exact quotient diagnostics, dual witnesses, two-stage coherence, and transport (CODA-FOC)*, 2026. https://doi.org/10.5281/zenodo.20492636
3. J. H. Carroll, *Finite Descent Authority: Packets, Pipelines, and Seven Halt Guards: A proposal is not a decision (CODA-FDA)*, 2026. DOI not asserted.
4. C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, 27(3):379–423, 1948. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x
5. P. Cousot and R. Cousot, "Abstract interpretation: a unified lattice model for static analysis of programs," in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pp. 238–252, 1977. https://doi.org/10.1145/512950.512973
6. X. Leroy, "Formal verification of a realistic compiler," *Commun. ACM*, 52(7):107–115, 2009. https://doi.org/10.1145/1538788.1538814